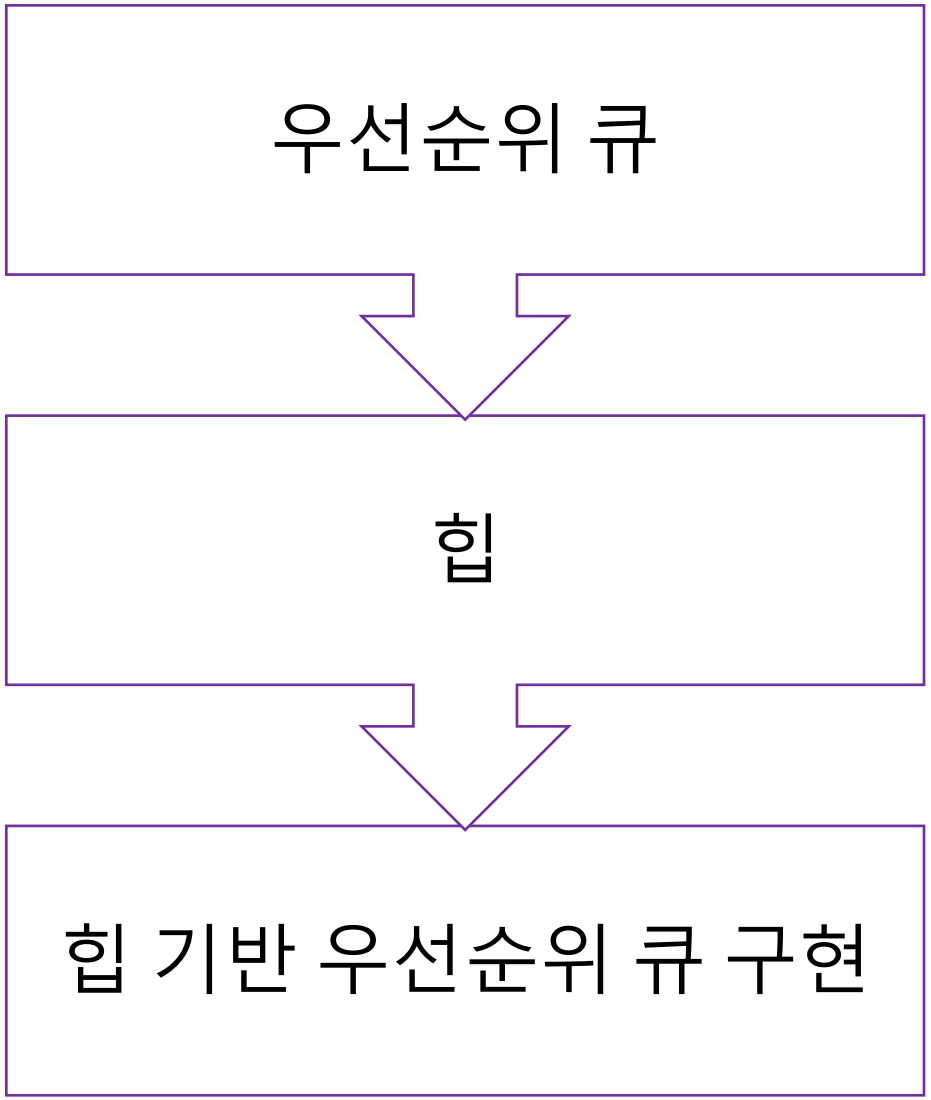
CHAPTER 07 우선순위 큐와 힙

Contents

- CHAPTER 07 – 우선순위 큐와 힙

SECTION 7-1 우선순위 큐
SECTION 7-2 힙
SECTION 7-3 힙 기반 우선순위 큐의 구현

CHAPTER 07 이 장의 학습 흐름

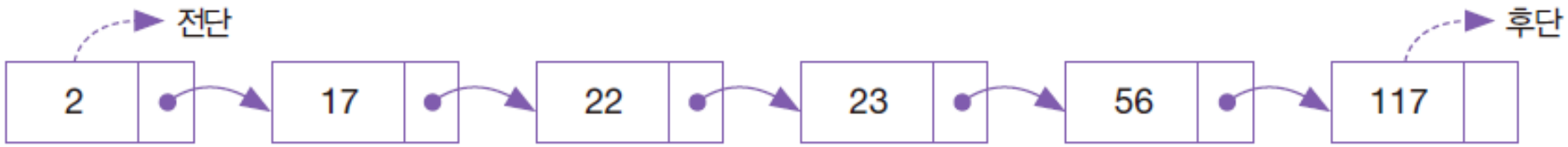


SECTION 7.1 우선순위 큐

- 우선순위 큐
  - 우선순위 속성을 갖는 데이터의 삽입(Enqueue)과 제거(Dequeue) 연산을 지원하는 ADT
  - 보통 큐는 먼저 들어온 요소가 무조건 먼저 나옴
  - 우선순위 큐는 새 요소에 우선순위를 부여해서 큐에 삽입하고 가장 높은 우선순위를 가진 요소부터 나옴

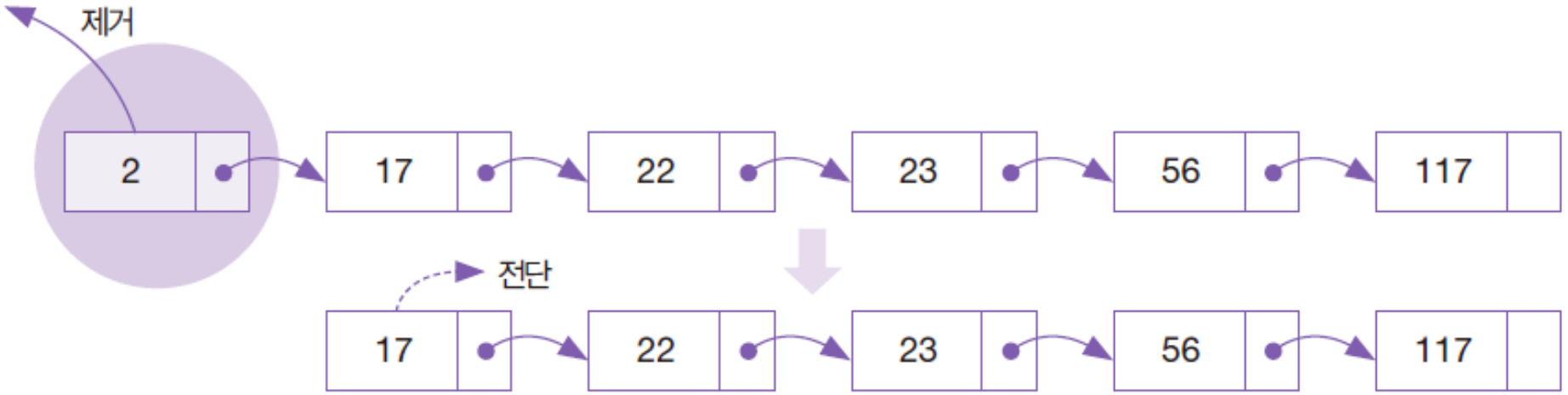
SECTION 7.1 우선순위 큐

1. 우선순위 큐의 삽입/제거 연산



- 노드 삽입 연산
  - 큐에 우선순위가 20인 요소를 삽입
    - 일단 우선순위 큐의 첫 요소부터 순차적으로 우선순위를 비교
    - 20은 2와 17보다 크고 22보다 작으므로 17과 22 사이에 삽입

- 노드 제거 연산
  - 큐의 제거(Dequeue) 연산은 전단만 제거해서 반환하면 완료

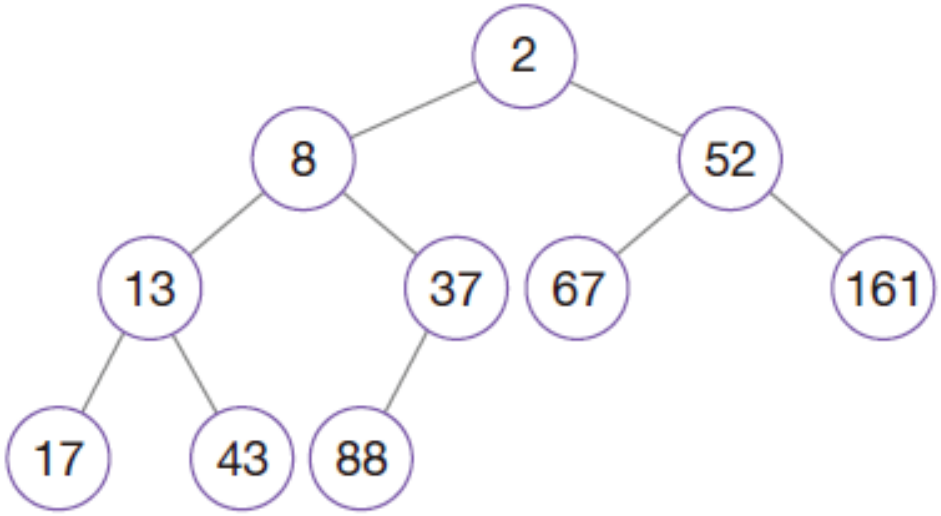


- **2. 우선순위 큐의 구현**
  - 힙(Heap)을 알아야 함

SECTION 7.2 힙

- 힙(Heap)
  - 힙 순서 속성(Heap Order Property)을 만족하는 완전 이진 트리
  - 프로그래밍 언어에서 말하는 자유 저장소(Free Store)의 힙과 다름
- 힙 순서 속성
  - 트리 내의 모든 노드가 부모 노드보다 커야 한다는 규칙
  - 뿌리 노드는 부모가 없으니 예외
  - 힙 내에서 어떤 노드를 찾으려면 모든 노드를 순회해야 하므로 이진 탐색 불가능
- 힙에 사용되는 연산
  - 노드를 삽입하는 연산
  - 뿌리 노드를 없애는 '최솟값 삭제' 연산

"힙에서 가장 작은 데이터를 갖는 노드는 뿌리 노드이다."



SECTION 7.2 힙

◦ 1. 힙의 삽입 연산

- 삽입 연산 단계

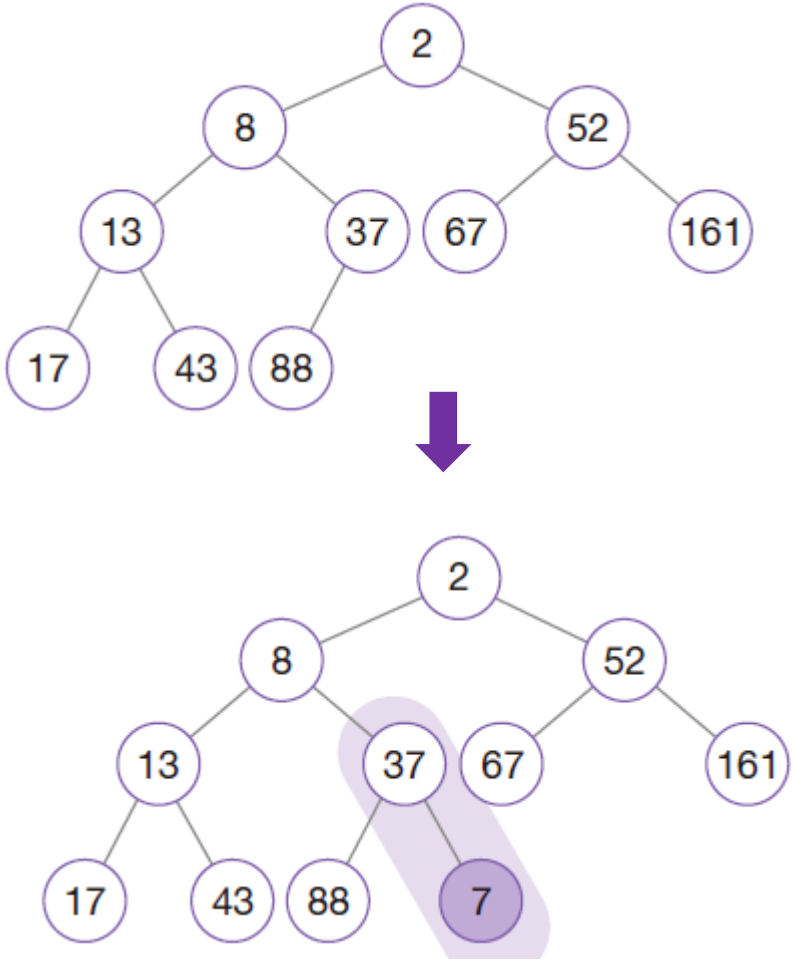
❶ 힙의 최고 깊이 가장 우측에 새 노드를 추가 (이때 힙은 완전 이진 트리를 유지하도록)

❷ 삽입한 노드를 부모 노드와 비교
- 삽입한 노드가 부모 노드보다 크면 제 위치에 삽입된 것이므로 연산을 종료
- 반대로 부모 노드보다 작은 경우에는 다음 단계를 진행

❸ 삽입한 노드가 부모 노드보다 작으면 부모 노드와 삽입한 노드의 위치를 서로 바꿈
- 바꾼 후에는 단계 ❷를 다시 진행

SECTION 7.2 힙

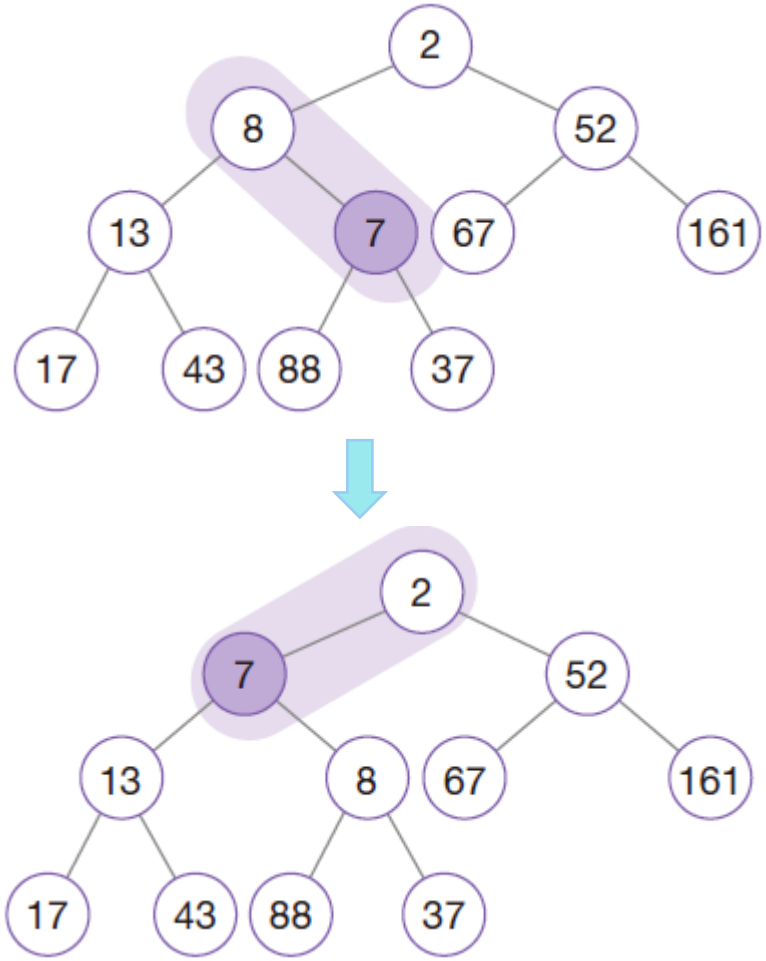
- 다음 그림과 같은 힙에 7 삽입



완전 이진 트리가 유지되도록 하며
노드 37 의 오른쪽 자식으로 추가

SECTION 7.2 힙

- 다음 그림과 같은 힙에 7 삽입



7은 37보다 작으므로 두 노드를 교환

7은 8보다 작으므로 두 노드를 교환

2. 힙의 최솟값 삭제 연산

- **힙의 최솟값 삭제 :** 뿌리 노드를 삭제
- 뿌리 노드를 삭제한 후 '힙 순서 속성'을 유지하기 위한 뒤처리 과정

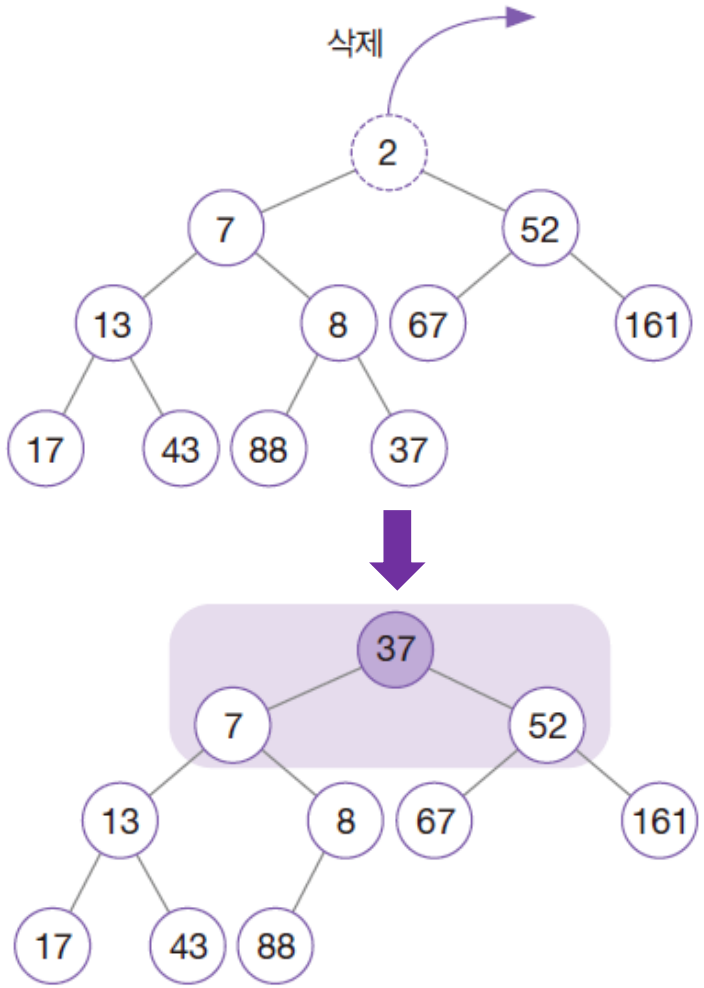
❶ 힙의 최고 깊이 가장 우측에 있던 노드를 뿌리 노드로 옮김
- 이때 힙 순서 속성이 파괴됩니다. 이를 복원하기 위한 작업을 다음 단계부터 시작

❷ 옮겨온 노드의 양쪽 자식을 비교하여 작은 쪽 자식과 위치를 교환
- 힙 순서 속성을 지키려면 부모 노드는 양쪽 자식보다 작은 값을 가져야 하기 때문임

❸ 옮겨온 노드가 더 이상 자식이 없는 잎 노드로 되거나 양쪽 자식보다 작은 값을 갖는 경우 삭제 연산을 종료
- 그렇지 않으면 단계 ❷를 반복

SECTION 7.2 힙

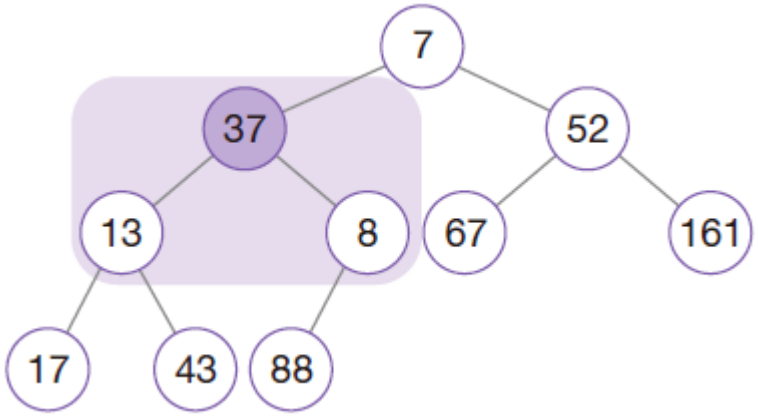
- 최솟값 노드, 즉, 뿌리 노드를 삭제



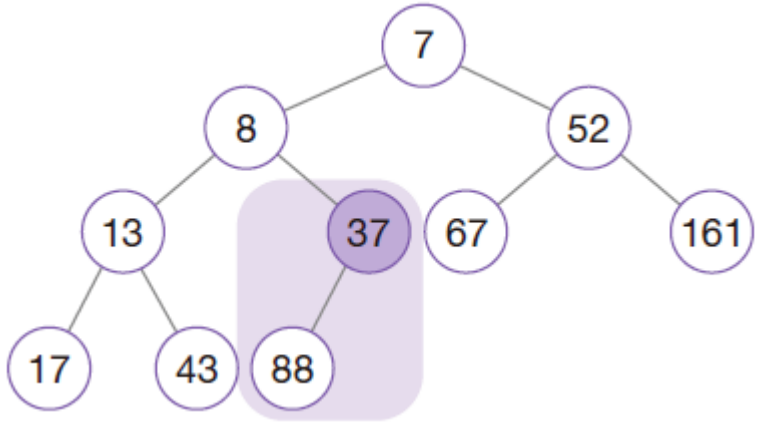
- 힙의 최고 깊이 가장 우측 노드를 삭제된 뿌리 노드가 있던 곳으로 옮김
- 뿌리 노드 37이 왼쪽 자식 노드 7보다 크므로 힙 순서 속성을 위반
- 뿌리 노드는 양쪽 자식 중 작은 값을 가진 노드와 자신을 교환해야 하므로 7과 37을 교환.

SECTION 7.2 힙

- 최솟값 노드, 즉, 뿌리 노드를 삭제



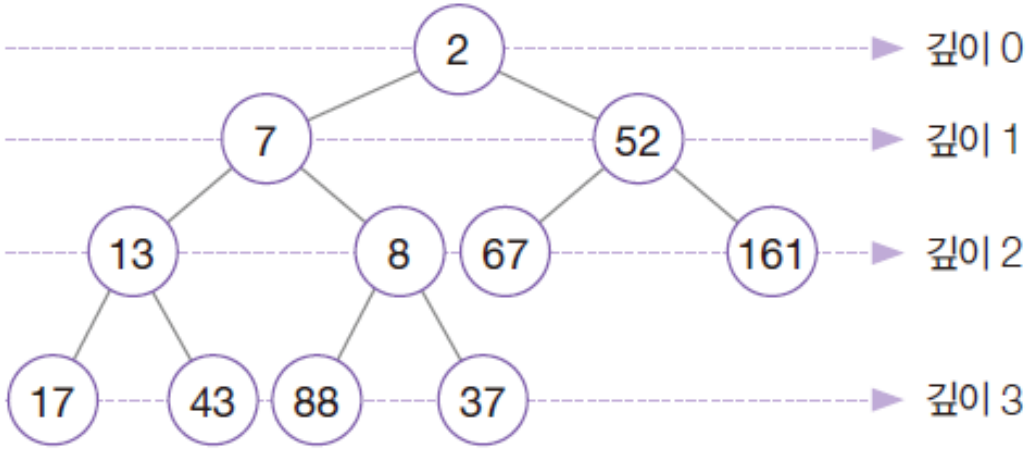
- 37이 양쪽 자식 13과 8보다 큼
- 13과 8 중에서 8이 더 작으므로 37 노드와 8 노드를 교환

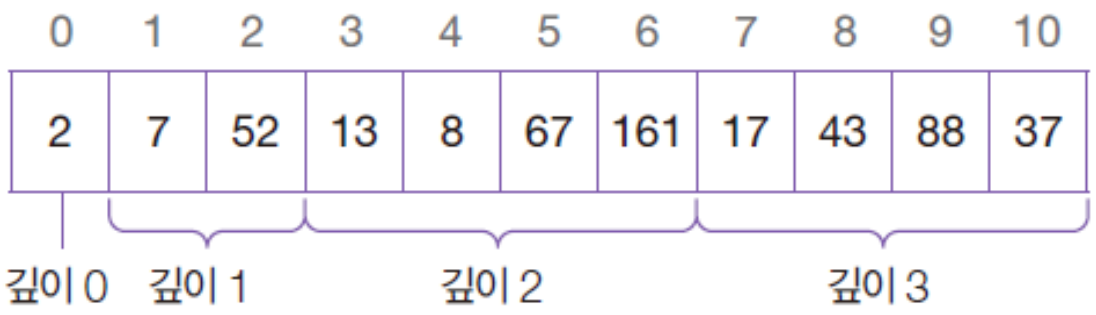


- 자식 노드는 88 하나뿐인데 37 노드는 이보다 더 작으므로 힙 순서 속성을 위반하지 않음
- 삭제 연산이 완료

3. 힙의 구현

- 힙의 구현에 사용되는 자료구조로는 링크드 리스트보다 배열이 효율적
- 배열을 이용한 완전 이진 트리 구현 규칙
  - 깊이 0의 노드는 배열의 0번 요소에 저장
  - 깊이 1의 노드(모두 2개)는 배열의 1~2번 요소에 저장
  - 깊이 2의 노드(모두 4개)는 배열의 3~6번 요소에 저장
  - 깊이 n의 노드(2n개)는 배열의 2n -1~2n+1-2번 요소에 저장





SECTION 7.2 힙

- 배열의 장점은 인덱스 정보만 있으면 힙의 각 노드 위치나 부모와 자식 관계 확인이 용이

- 배열 기반의 완전 이진 트리 처리 공식

  k번 인덱스에 위치한 노드의 양쪽 자식 노드들이 위치한 인덱스

  - 왼쪽 자식 노드: 2k + 1
  - 오른쪽 자식 노드: 2k + 2

  k번 인덱스에 위치한 노드의 부모 노드가 위치한 인덱스: (k-1)÷2의 몫

SECTION 7.2 힙

- 자료구조 선언
  - 2개의 구조체를 가진 힙 구현
    - 힙의 각 노드를 표현하는 HeapNode 구조체
    - 힙 자체를 표현하는 Heap 구조체
      - Capacity 필드 - 힙의 용량, 즉 Nodes 배열의 크기
      - UsedSize – 실제 배열 안에 들어 있는 큐 요소의 수

```
typedef int ElementType;

typedef struct tageHeapNode {
    ElementType Data;
} HeapNode;

typedef struct tagHeap {
    HeapNode* Nodes;
    int Capacity;
    int UsedSize;
} Heap;
```

- 삽입 연산 구현
  - HEAP_Insert ( )

```
void  HEAP_Insert( Heap* H, ElementType NewData )
{
    int CurrentPosition = H->UsedSize;
    int ParentPosition  = HEAP_GetParent( CurrentPosition );

    if ( H->UsedSize == H->Capacity )
    {
        H->Capacity *= 2;
        H->Nodes = (HeapNode*)
                   realloc( H->Nodes, sizeof( HeapNode ) * H->Capacity );
    }

    H->Nodes[CurrentPosition].Data = NewData;
```

UsedSize가 Capacity에 도달하면 Capacity를 2배로 늘립니다.

▶ 다음 쪽에 이어짐

◀ 앞쪽에 이어

```
    //  후속 처리
    while ( CurrentPosition > 0
        && H->Nodes[CurrentPosition].Data < H->Nodes[ParentPosition].Data )
    {
        HEAP_SwapNodes( H, CurrentPosition, ParentPosition );

        CurrentPosition = ParentPosition;
        ParentPosition  = HEAP_GetParent( CurrentPosition );
    }

    H->UsedSize++;
}
```

- 최솟값 삭제 연산 구현
  - Heap_DeleteMin( )

```
void      HEAP_DeleteMin( Heap* H, HeapNode* Root )
{
    int ParentPosition = 0;
    int LeftPosition   = 0;
    int RightPosition  = 0;

    memcpy(Root, &H->Nodes[0], sizeof(HeapNode));
    memset(&H->Nodes[0], 0, sizeof(HeapNode));

    H->UsedSize--;
    HEAP_SwapNodes( H, 0, H->UsedSize );
```

Root에 최솟값을 저장합니다.

힙의 첫 번째 요소에 마지막 요소의 값을 복사합니다.

▶ 다음 쪽에 이어짐

◀ 앞쪽에 이어

```
    LeftPosition  = HEAP_GetLeftChild( 0 );
    RightPosition = LeftPosition + 1;

    while ( 1 )
    {
        int SelectedChild = 0;

        if ( LeftPosition >= H->UsedSize )
            break;

        if ( RightPosition >= H->UsedSize )
        {
            SelectedChild = LeftPosition;
        }
        else {
```

힙 순서 속성을 만족할 때까지 자식 노드와의 교환을 반복합니다.

▶ 다음 쪽에 이어짐

◀ 앞쪽에 이어

```
            if ( H->Nodes[LeftPosition].Data > H->Nodes[RightPosition].Data)
                SelectedChild = RightPosition;
            else
                SelectedChild = LeftPosition;
        }

        if ( H->Nodes[SelectedChild].Data < H->Nodes[ParentPosition].Data )
        {
            HEAP_SwapNodes(H, ParentPosition, SelectedChild);
            ParentPosition = SelectedChild;
        }
        else
            break;
```

▶ 다음 쪽에 이어짐

◀ 앞쪽에 이어

```
        LeftPosition  = HEAP_GetLeftChild(ParentPosition);
        RightPosition = LeftPosition + 1;
    }

    if ( H->UsedSize < ( H->Capacity / 2 ) )
    {
        H->Capacity /= 2;
        H->Nodes =
            (HeapNode*) realloc( H->Nodes, sizeof( HeapNode ) * H->Capacity );
    }
}
```

- 4. 힙 예제 프로그램
  - 3개의 파일
    - **Heap.h, Heap.c** – 힙의 각종 연산 수행 함수들을 선언 및 구현
    - **Test_Heap.c** – 함수들을 테스트

실행 결과

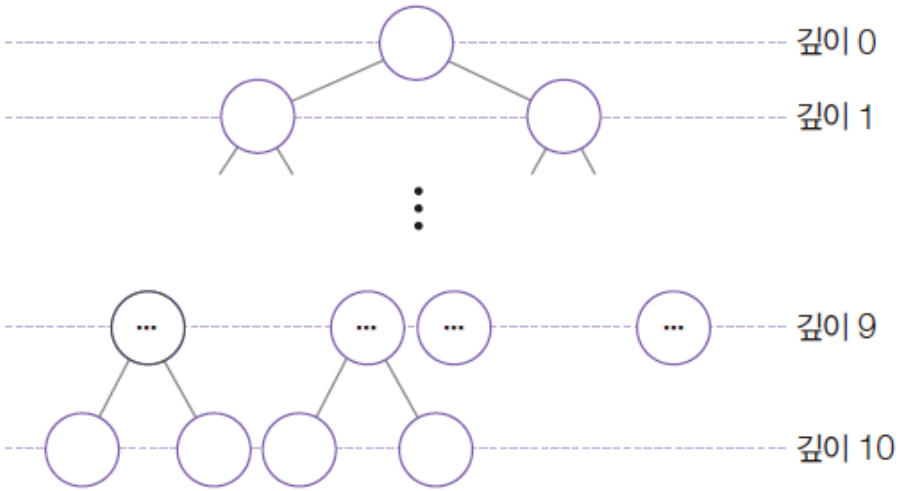
```
size : 4
12 16 75 87 34 111
16 34 75 87 111
34 87 75 111
75 87 111
87 111
111
```

3. 힙 기반 우선순위 큐의 구현

- 우선순위 큐 예제 프로그램
  - 노드 구조체에 Priority 필드 추가
  - 노드 구조체의 Data 필드 자료형을 void*로 변경
  - 삽입, 삭제 함수의 이름을 각 HEAP_Insert ( ), HEAP_DeleteMin ( )에서 PQ_Enqueue( ), PQ_Dequeue( )로 변경
- 3개의 파일
  - **PriorityQueue.h, PriorityQueue.c** - 우선순위 큐의 자료구조를 이루는 구조체와 각 연산을 수행하는 함수들을 정의
  - **Test_PriorityQueue.c** – 함수 테스트

실행 결과

```
큐에 남아 있는 작업의 수 : 6
작업명 : 고객미팅 (우선순위:12)
작업명 : 코딩 (우선순위:34)
작업명 : 디버깅 (우선순위:35)
작업명 : 문서작성 (우선순위:45)
작업명 : 이닦기 (우선순위:66)
작업명 : 커피타기 (우선순위:87)
```

연습 문제

- 01 힙은 이진 트리로 구성되어 있다. 그렇다면 힙에서 가장 작은 값을 가진 노드는 어떤 노드일까?
  - ① 가장 좌측 하단 노드
  - ② 가장 우측 하단 노드
  - ③ 뿌리 노드
- 02 힙은 개념적으로 이진 트리로 구성되어 있지만 우리는 배열을 이용해서 구현했다. 깊이 0의 노드는 배열의 0번 요소에, 깊이 1의 노드는 배열의 1~2번 요소에 저장하도록 구현. 그렇다면 힙의 트리가 포화 이진 트리라고 했을 때 깊이 10에 있는 노드는 배열의 몇 번부터 몇 번까지의 요소에 저장될까?